

Lab 2 - Part B: LeNet-5 Architecture on MNIST with Hyperparameter Tuning

Name: Arnav Narula Reg No: 2547115

```
In [1]: import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import layers, models, optimizers
```

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
 I0000 00:00:1790258061.441067 178048 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.
 I0000 00:00:1790258061.489824 178048 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
 To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
 WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
 I0000 00:00:1790258062.550164 178048 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

Load and Preprocess MNIST Dataset

```
In [2]: (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# LeNet-5 expects 32x32 images, MNIST is 28x28, so we pad it
x_train = np.pad(x_train, ((0,0),(2,2),(2,2)), 'constant')
x_test = np.pad(x_test, ((0,0),(2,2),(2,2)), 'constant')

# Expand dims and normalize
x_train = np.expand_dims(x_train, axis=-1).astype('float32') / 255.0
x_test = np.expand_dims(x_test, axis=-1).astype('float32') / 255.0

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

print('x_train shape:', x_train.shape)
print('y_train shape:', y_train.shape)
```

```
x_train shape: (60000, 32, 32, 1)
y_train shape: (60000, 10)
```

Define LeNet-5 Architecture

```
In [3]: def build_lenet5(learning_rate=0.001):
        model = models.Sequential()
        model.add(layers.Conv2D(6, kernel_size=(5, 5), strides=(1, 1), activation='tanh'))
        model.add(layers.AveragePooling2D(pool_size=(2, 2), strides=(2, 2), padding='valid'))
        model.add(layers.Conv2D(16, kernel_size=(5, 5), strides=(1, 1), activation='tanh'))
        model.add(layers.AveragePooling2D(pool_size=(2, 2), strides=(2, 2), padding='valid'))
        model.add(layers.Flatten())
        model.add(layers.Dense(120, activation='tanh'))
        model.add(layers.Dense(84, activation='tanh'))
        model.add(layers.Dense(10, activation='softmax'))

        optimizer = optimizers.Adam(learning_rate=learning_rate)
        model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
        return model

model = build_lenet5()
model.summary()
```

/home/arden/Coding/AMI_Classwork/.venv/lib/python3.12/site-packages/keras/src/layer/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
W0000 00:00:1790258068.144411 178048 gpu_device.cc:2365] Cannot dlopen some GPU libraries. Please make sure the missing libraries mentioned above are installed properly if you would like to use GPU. Follow the guide at <https://www.tensorflow.org/install/gpu> for how to download and setup the required libraries for your platform. Skipping registering GPU devices...

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 6)	156
average_pooling2d (AveragePooling2D)	(None, 14, 14, 6)	0
conv2d_1 (Conv2D)	(None, 10, 10, 16)	2,416
average_pooling2d_1 (AveragePooling2D)	(None, 5, 5, 16)	0
flatten (Flatten)	(None, 400)	0
dense (Dense)	(None, 120)	48,120
dense_1 (Dense)	(None, 84)	10,164
dense_2 (Dense)	(None, 10)	850

Total params: 61,706 (241.04 KB)

Trainable params: 61,706 (241.04 KB)

Non-trainable params: 0 (0.00 B)

Hyperparameter Tuning (Learning Rate)

```
In [4]: learning_rates = [0.01, 0.001, 0.0001]
        histories = []

        for lr in learning_rates:
            print(f'\n--- Training with Learning Rate: {lr} ---')
            model = build_lenet5(learning_rate=lr)
            history = model.fit(x_train, y_train, batch_size=128, epochs=5, validation_data=
            histories.append((lr, history))
```

--- Training with Learning Rate: 0.01 ---

Epoch 1/5

469/469  **6s** 11ms/step - accuracy: 0.9298 - loss: 0.2320 - val_accuracy: 0.9632 - val_loss: 0.1219

Epoch 2/5

469/469  **7s** 14ms/step - accuracy: 0.9614 - loss: 0.1289 - val_accuracy: 0.9583 - val_loss: 0.1377

Epoch 3/5

469/469  **7s** 14ms/step - accuracy: 0.9658 - loss: 0.1125 - val_accuracy: 0.9694 - val_loss: 0.0992

Epoch 4/5

469/469  **6s** 12ms/step - accuracy: 0.9698 - loss: 0.1001 - val_accuracy: 0.9676 - val_loss: 0.1054

Epoch 5/5

469/469  **6s** 13ms/step - accuracy: 0.9700 - loss: 0.0987 - val_accuracy: 0.9669 - val_loss: 0.1139

--- Training with Learning Rate: 0.001 ---

Epoch 1/5

469/469  **8s** 15ms/step - accuracy: 0.9026 - loss: 0.3414 - val_accuracy: 0.9532 - val_loss: 0.1602

Epoch 2/5

469/469  **7s** 15ms/step - accuracy: 0.9628 - loss: 0.1222 - val_accuracy: 0.9698 - val_loss: 0.0957

Epoch 3/5

469/469  **6s** 13ms/step - accuracy: 0.9757 - loss: 0.0790 - val_accuracy: 0.9771 - val_loss: 0.0713

Epoch 4/5

469/469  **7s** 14ms/step - accuracy: 0.9824 - loss: 0.0585 - val_accuracy: 0.9810 - val_loss: 0.0572

Epoch 5/5

469/469  **6s** 13ms/step - accuracy: 0.9859 - loss: 0.0465 - val_accuracy: 0.9830 - val_loss: 0.0499

--- Training with Learning Rate: 0.0001 ---

Epoch 1/5

469/469  **8s** 15ms/step - accuracy: 0.7941 - loss: 0.8975 - val_accuracy: 0.8931 - val_loss: 0.4154

Epoch 2/5

469/469  **7s** 15ms/step - accuracy: 0.8961 - loss: 0.3689 - val_accuracy: 0.9146 - val_loss: 0.3099

Epoch 3/5

469/469  **6s** 13ms/step - accuracy: 0.9143 - loss: 0.2948 - val_accuracy: 0.9274 - val_loss: 0.2579

Epoch 4/5

469/469  **6s** 13ms/step - accuracy: 0.9265 - loss: 0.2482 - val_accuracy: 0.9365 - val_loss: 0.2191

Epoch 5/5

469/469  **6s** 12ms/step - accuracy: 0.9372 - loss: 0.2119 - val_accuracy: 0.9458 - val_loss: 0.1895

Visualize Tuning Results

```
In [5]: plt.figure(figsize=(12, 5))
```

```

# Plot Validation Accuracy
plt.subplot(1, 2, 1)
for lr, history in histories:
    plt.plot(history.history['val_accuracy'], label=f'LR = {lr}')
plt.title('Validation Accuracy by Learning Rate')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot Validation Loss
plt.subplot(1, 2, 2)
for lr, history in histories:
    plt.plot(history.history['val_loss'], label=f'LR = {lr}')
plt.title('Validation Loss by Learning Rate')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()

```

